

Create a Digital Twin Using Python

In this guide, we will demonstrate how to build a basic yet effective Digital Twin using Python. A Li-ion battery will serve as our tangible asset. This Digital Twin will allow us to assess and forecast battery performance, and it can be incorporated into any virtual asset management process.

Virtual systems and digital twins

Digital twins play a vital role in Industry 4.0. The fundamental concept is to recreate physical assets within a virtual environment to simulate their behavior. Envision a water pump located within a city's network of water pipes. Water pumps utilize energy and convert it into water flow and pressure. The term “replica” denotes the development of a virtual object that can imitate this behavior.

This pump is a component of the complete water pipeline network system, which includes pipes, valves, meters, and additional accessories or subsystems. All these subsystems are interconnected. Virtualizing the system means applying the same process to all subsystems. Subsequently, we can model the complete system, along with its dependencies. The aim of a virtual system is to replicate the physical world to make changes, assess performance, or forecast situations involving this asset or subsystem (such as in maintenance activities).

A digital twin, as earlier described, is a virtual entity that symbolizes a subsystem. This “twin” is anticipated to react to input variables in the same manner as its “physical twin” does. This digital object needs to incorporate a model to achieve this. The key characteristic of the digital twin is a model capable of replicating “physical” behavior within a digital environment. Please remember that when we use the term “physical,” we refer to any entity in the real world (this could be a Li-ion battery, a water pump, an individual, a city, or a cat). Anything that can be represented can be virtualized (figure 1).

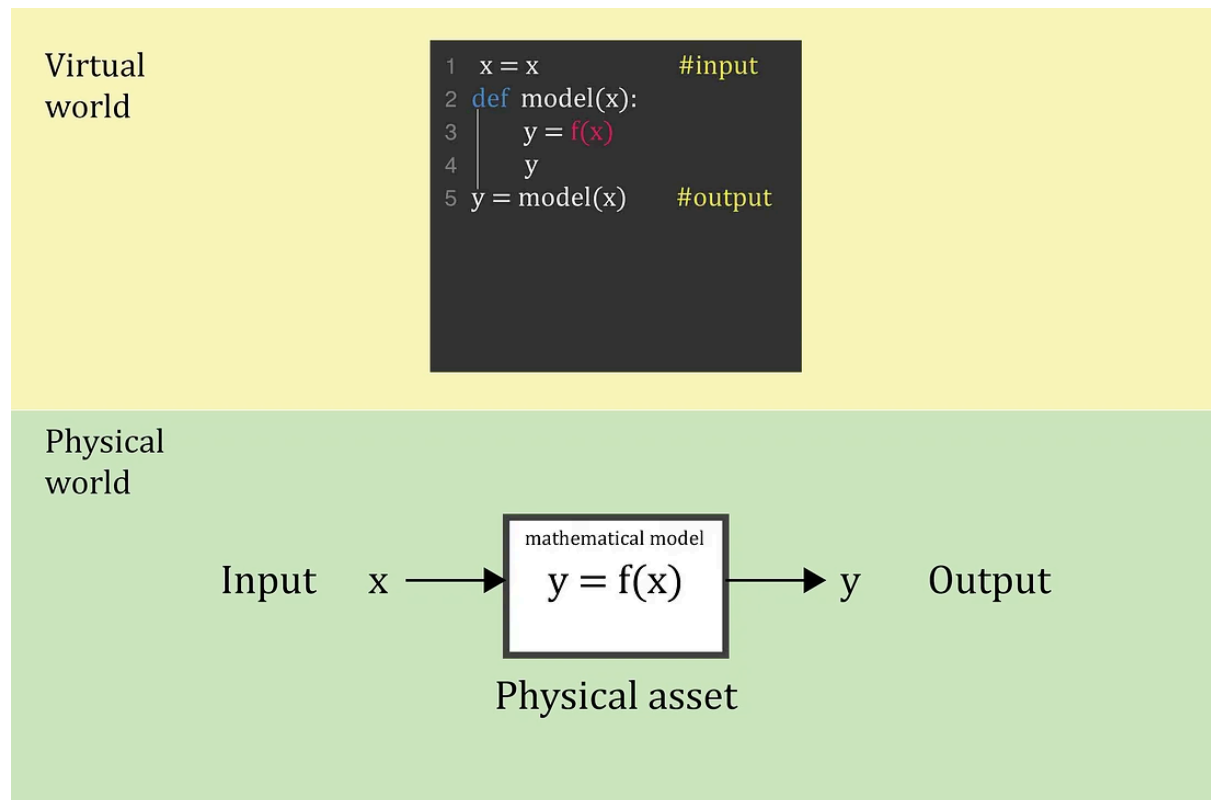


Figure 1. From a physical world to a virtual world through a model.

Digital model of Lithium-ion battery

Rechargeable lithium-ion batteries represent an advanced battery technology that depends on lithium ions within its electrochemical processes. These batteries serve as valuable resources for uses like electric vehicles and energy storage within electric distribution systems, as well as for portable technological devices. The most significant factor regarding these batteries is their aging cost. Following numerous charge-discharge cycles, battery cells deteriorate, leading to a decrease in charging capacity. This occurrence has been a crucial focus in the advancement of more enduring batteries. The modeling of this subject has also been a debated issue. Battery deterioration significantly affects the planning and operations of the technologies mentioned above.

The science behind the decline in battery life is fairly complex. Several semi-empirical models for the degradation of lithium-ion batteries, aimed at assessing the loss of battery cell life, have been suggested in recent years (Chu et al., 2018; Laresgoiti et al., 2015). Batteries at the end of their life are typically defined as those that provide only 80% of their maximum rated capacity. This decline can be expressed in the following way using one of the empirical models (Chu et al., 2018):

$$L = 1 - (1 - L')e^{f_d}$$

Equation 1. Battery life-time.

where L represents the battery lifespan, L' denotes the starting battery lifespan, and f_d indicates a linearized degradation rate for each unit of time and cycle (Chu et al., 2018). This rate can also be expressed as a function of discharge time - t , the depth of discharge cycle - δ , the average state of charge during the cycle - σ , and the temperature of the cell - T_c .

$$f_d = f_d(t, \delta, \sigma, T_c)$$

Equation 2. Linearized degradation rate.

Experimental data

Our model needs to predict battery longevity using the data we've gathered. Nevertheless, we aim to contrast this model with measurements obtained to assess the precision of our model. Let us take a moment to stop here.

Numerous digital twins do not rely on precise physical models and solely utilize experimental data sets along with machine learning techniques. This is a really awesome way of accomplishing tasks. For instance, they can create a model employing a deep neural network to understand the actual dynamics of the data. Nonetheless, extending this model to different objects requires handling a substantial number of samples to achieve a dependable general model. Conversely, when using physical models, we need less data to create an accurate general model.

We will utilize actual data from charge-discharge cycles in lithium-ion batteries to create our digital twin. We will utilize Li-ion Battery Aging Datasets sourced from NASA Ames Prognostics Center of Excellence (PCoE). This dataset will be useful for assessing the accuracy of our physical model and enhancing it. We will utilize the information linked to battery number 5. We will graph the 'Capacity' attribute against the number of cycles and contrast it with our physical model. Before we start, we will replace the variable L (battery life-time) with C (battery capacity). Equation 1 can be expressed in the following manner:

$$C = C_0 e^{f_d}$$

where C is the battery capacity and C_0 is the initial battery capacity. For f_d we have used the following approximation:

$$f_d = k \frac{i \cdot T_c}{t_i}$$

where i is the charge-discharge cycle, T_c the temperature measured in the cell during the cycle, t_i is the discharging time and k and empirical constant with value 0.13 .

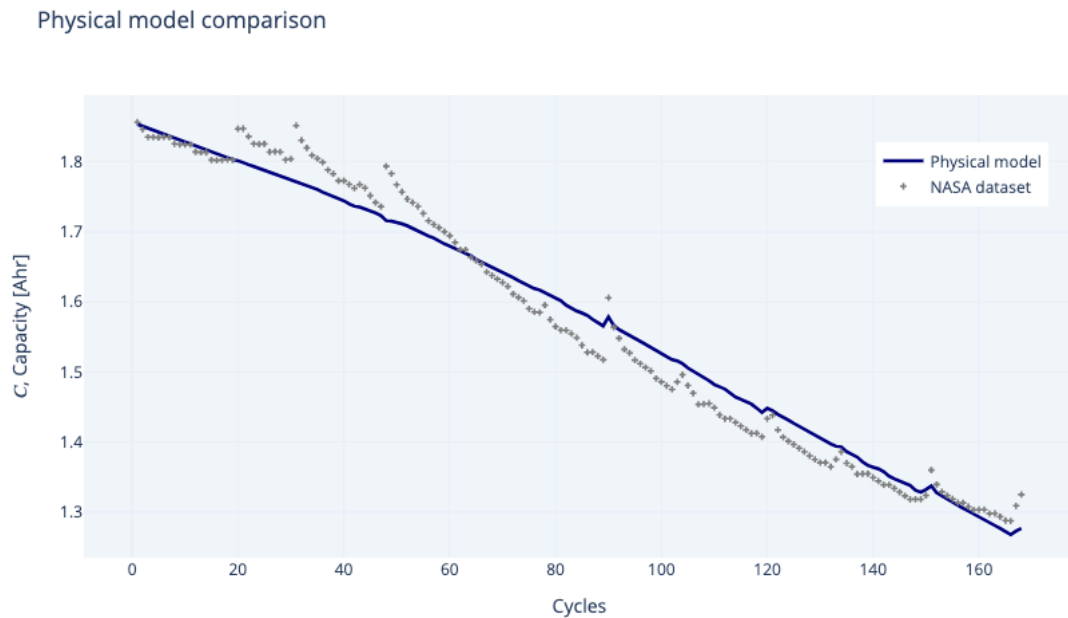


Figure 2. Comparison of experimental data with our semi-empirical model. Image by author.

Building an hybrid digital twin

We could our “model” to directly create a digital twin. But, in order to apply Machine learning, we are going to use it to improve our model using the experimental data from NASA dataset. Our proposal is shown in figure 3.

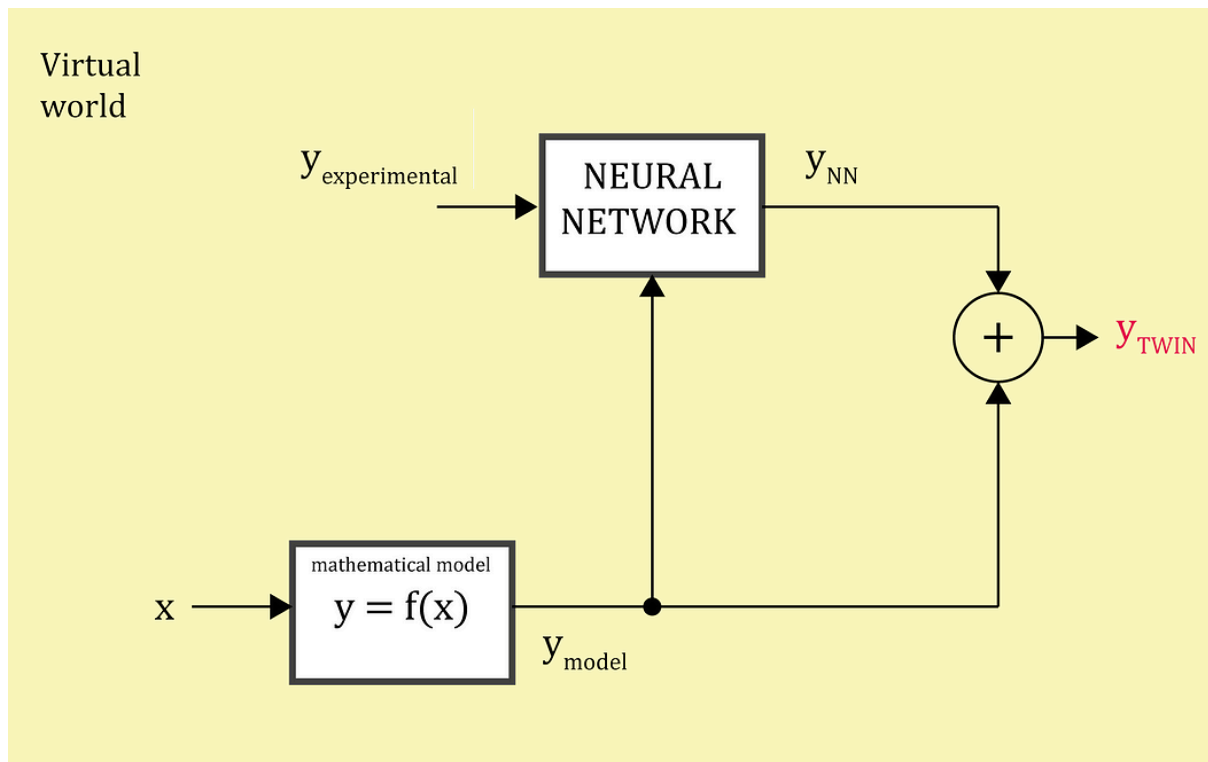


Figure 3. Creating an hybrid digital twin. Image by author.

We are improving our mathematics using experimental data and our model's output using a NN.

```
#Define inputs and outputs
# input: the simulation capacity
X_in = (dfb['C. Capacity'])
# output: difference between experimental values and simulation
X_out = (dfb['Capacity']) - (dfb['C. Capacity'])
X_in_train, X_in_test, X_out_train, X_out_test = train_test_split(X_in, X_out, test_size=0.33)
```

We are using a very simple neural network:

```
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(1,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))
```

Compilation:

```
epochs = 100
loss = 'mse'
```

```

model.compile(optimizer = SGD(learning_rate=0.001),
              loss=loss,
              metrics=['mae'], #Mean Absolute Error
)
history = model.fit(X_in_train,
                  X_out_train,
                  shuffle=True,
                  epochs=epochs,
                  batch_size=20,
                  validation_data=(X_in_test, X_out_test),
                  verbose=1)

```

The comparison's result:

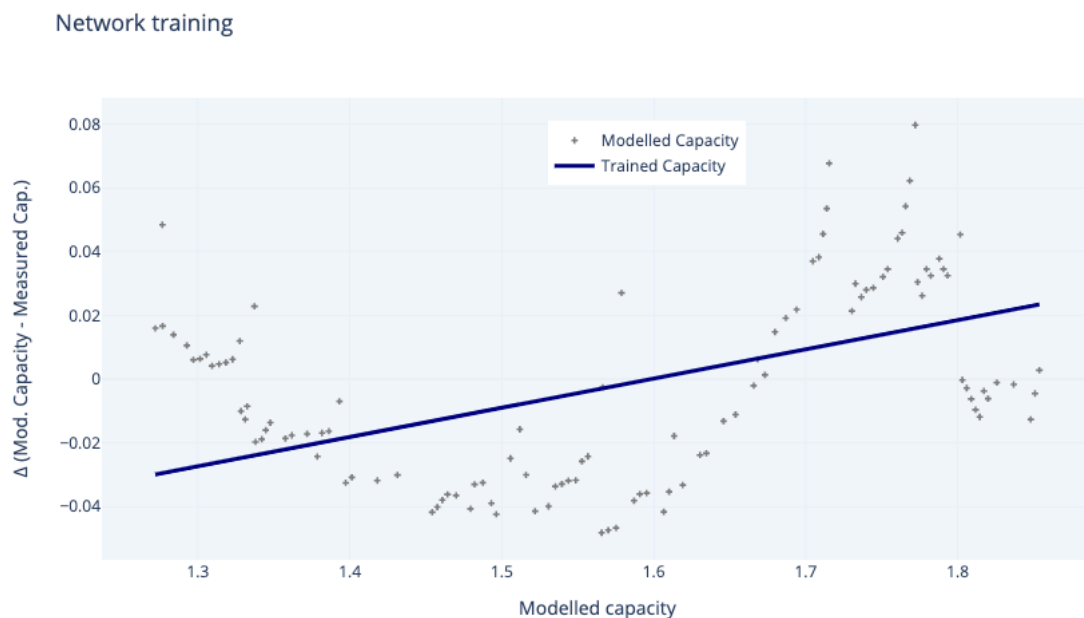


Figure 4. Model results comparison with experimental result. Image by author.

As we can see in figure 4, NN learns a basic understanding of the differences between our mathematical model and the experimental results. Now we can add this learning to our mathematical model to improve it:

```

# Our digital twin by improving our model with experimental data
X_twin = X_in + model.predict(X_in).reshape(-1)

```

And that's all. We possess our digital twin (or hybrid twin) in which:

We have developed a physical model (or mathematical model).

We have contrasted our model with experimental findings.

Through this comparison, we enhanced our model.

Ultimately, we possess a “twin” of this battery within our virtual environment that is anticipated to function similarly to its real-world equivalent. Subsequently, we aim to compare our mathematical model with that of our digital twin:

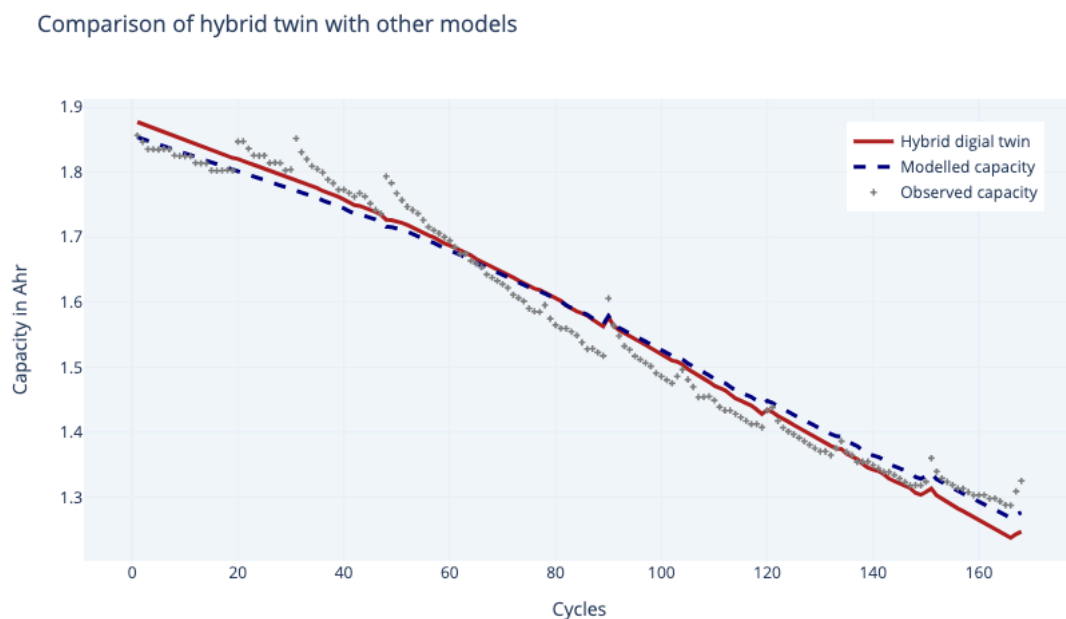


Figure 5: Final comparison of mathematical model with “hybrid” digital twin. Image by author.

Figure 5 demonstrates how our digital twin has slightly enhanced our model. The advantage of a hybrid digital twin is that it enables us to use a semi-empirical mathematical model and enhance it with experimental data. It surpasses both a specifically mathematical model, as it allows for enhancements, and an ML model, due to being more versatile (which means it could be used, for instance, on different batteries achieving greater precision than a standalone ML model).

Prediction

Using our digital twin, we can forecast operations for the Li-ion battery.

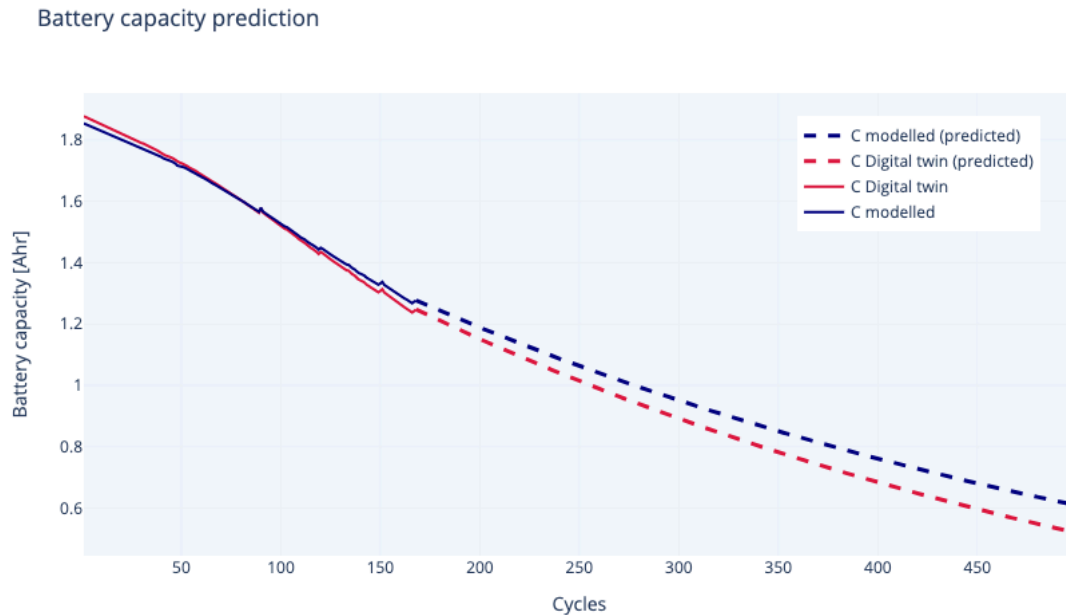


Figure 6. Predictions using digital twin and mathematical model. Image by author.

Conclusion:

Digital twins represent a developing subject in the realm of Industry 4.0. We've demonstrated how to utilize Python to create a basic digital twin. When we possess experimental data sets, we can demonstrate how "hybrid" digital twins offer a more authentic method to virtualize assets. Moreover, a smaller amount of data is needed to create trustworthy models. We can incorporate additional future experimental data measurements into our digital twin to further enhance our mathematical model.

source:

<https://towardsdatascience.com/how-to-build-a-digital-twin-b31058fd5d3e>

For Code testing please consider the material added in the archive of this folder:

[discharge.csv](#)

[digitalTwin_Li_ION.ipynb](#)